LLM-Driven NPC Scheduling in Structured Game Worlds

Peter Husen | Student Number: 233900
Breda University of Applied Sciences
Supervisor: Edirlei Soares de Lima
June 8th 2026

Abstract

Non-player character behaviour in games is traditionally defined through hand-authored rule-based systems such as finite state machines and behaviour trees, which are constrained to situations their authors anticipated and fail to adapt when players act outside expected scenarios. This paper proposes a method for using large language models (LLMs) to generate NPC action schedules at runtime, replacing pre-authored transition logic with a pipeline driven by four inputs: a structured world state encoding current locations, characters, items and their conditions; an action library defining available NPC actions; natural language world rules specifying action preconditions; and a character description defining the NPC's personality, role and goals. For each NPC, the LLM generates a candidate action schedule in structured format, which is validated against a deterministic rule-based validator before execution. The method is implemented in a zombie apocalypse survival simulation and evaluated across four LLMs, five world state scenarios, and two levels of rule specification, over 400 schedule generations per model. Schedules are assessed on rule validity, measured by the deterministic validator, and behavioural quality, assessed through qualitative content analysis of 640 sampled schedules. Results show that GPT-OSS-120B achieved the highest rule validity rate at 96.8%, followed by Qwen3.6-27B at 94.2%, Qwen3.5-122B at 87.5%, and Llama-3.3-70B-Instruct at 14.7%. The dominant failure mode was world-state reasoning rather than rule complexity, and ruleset specification depth had no statistically significant effect on validity for any model. These findings demonstrate that LLM-driven schedule generation is a feasible approach to autonomous NPC behaviour, with performance varying substantially across model families.

Keywords: NPC behaviour generation; large language models; schedule generation; game AI; procedural content generation

1. Introduction

Modern game worlds increasingly offer players substantial freedom to act, explore, and experiment. Yet the characters inhabiting those worlds are typically governed by systems built on a fixed assumption: that every situation the player might create can be anticipated in advance. Under the dominant paradigm, a developer defines the states an NPC can occupy, the conditions that trigger transitions between them, and the actions available in each state (Millington & Funge, 2009). Behaviour trees improved on finite state machines by making this process more modular, but the underlying assumption remained unchanged. As player freedom grows, the gap between what the system was designed to handle and what the player can do widens, and no hand-authored system can remain believable at the limit of that freedom. The deeper problem is structural: rule-based systems can only respond to situations their authors imagined.

This paper investigates a fundamentally different approach, one where NPC behaviour is not selected from a pre-authored graph but generated at runtime by reasoning over the current state of the game world. Large language models, with their capacity to interpret structured context and produce contextually appropriate outputs, present a candidate method for this kind of generation. Rather than encoding behaviour as explicit transitions, a developer defines an NPC through a natural language description of who the character is, their role, their goals, and how they prioritise those goals when they conflict. The model then generates sequential behaviour at runtime by reasoning over the shared world state and a set of natural language rules governing what actions are valid under what conditions. Character behaviour emerges from language understanding rather than a decision graph a developer must build and maintain.

This approach introduces a fundamental trade-off. Hand-authored systems guarantee that behaviour stays within defined bounds, because the bounds are the system. A language model does not offer that guarantee: it may reason incorrectly about the world state, overlook a constraint, or produce a plausible-sounding sequence that violates a game rule. Correctness becomes a matter of empirical reliability rather than logical necessity.

Whether that reliability is sufficient for practical use, and which models achieve it, are the central empirical questions this paper addresses. The evaluation is structured around four concerns: first, whether the pipeline produces executable schedules reliably, combining a structured world state, natural language world rules, an action library, and a character description into action sequences that clear deterministic validation. Second, whether valid schedules are also good schedules, in the sense of reflecting character-appropriate behaviour and situational awareness rather than merely passing rule checks. Third, what the characteristic failure modes are when the pipeline breaks down, and whether those failures are attributable to rule complexity, world-state reasoning, or something else. Fourth, what the performance characteristics of the approach look like in terms of generation latency, token usage, and schedule length, factors that bear directly on whether the method is viable in a real deployment context.

The method is implemented and evaluated in a zombie apocalypse survival simulation built as a 2D top-down RPG. Four large language models are compared to the validity of their generated NPC behaviour and on its qualitative plausibility, across five distinct world state scenarios and two levels of rule specification. Results show substantial variation across models, with the best-performing model achieving a validity rate of 96.8% and the weakest at 14.7%. The dominant failure mode across models was incorrect reasoning about the world state rather than the complexity of the rules themselves, the level of rule specification had no statistically significant effect on validity for any model.

The remainder of this paper is structured as follows. Section 2 reviews related work on LLM-driven NPC systems, autonomous agent architectures, and evaluation methods. Section 3 describes the proposed scheduling system, including its inputs, generation pipeline, and validation approach. Section 4 presents the game prototype in which the system is implemented. Section 5 describes the evaluation methodology, reports results, and discusses findings in relation to the research questions. Section 6 concludes with practical implications and directions for future work.

2. Related Work

The dominant approach to NPC behaviour throughout game development history has been the finite state machine (FSM) (Millington & Funge, 2009). In an FSM, an NPC exists in exactly one state at a time, such as patrolling, chasing, or attacking, and transitions between states when defined conditions are met. While FSMs are easy to write and fast to execute, they are notoriously difficult to maintain at scale, and complex FSMs can require thousands of lines of code with programmers responsible for writing the AI behaviour of each individual character (Millington & Funge, 2009). The fundamental problem is structural: as the complexity of agent behaviour increases, the state machine grows uncontrollably and it becomes difficult to express composed behaviours that span multiple states.

Behaviour trees emerged as the dominant industry response to these limitations. Rather than dispersing transition logic across individual states, a behaviour tree organises behaviour into a hierarchical structure of composable subtrees that can be rearranged or replaced independently, significantly improving modularity and reducing the cost of adding new behaviours (Iovino et al., 2022). Despite this improvement, behaviour trees remain fundamentally hand-authored: a designer must still enumerate every meaningful behaviour and define the conditions under which it applies. Both FSMs and behaviour trees share the same underlying assumption, that the full space of relevant player interactions can be anticipated and encoded in advance, and it is this assumption that LLM-driven approaches attempt to move beyond.

A third classical approach relevant to the present work is goal-oriented action planning (GOAP), introduced by Orkin (2004) and applied in games such as F.E.A.R. (Orkin, 2006). Rather than defining transitions between states, GOAP allows an NPC to specify a goal and automatically constructs a sequence of actions satisfying it by reasoning over formally specified preconditions and effects. This is structurally the closest classical analogue to what the present system does: both derive a sequence of actions from a goal and a world description. The key

difference is that GOAP requires a developer to formally specify every action's preconditions and effects in a structured planning domain, whereas the proposed method expresses these constraints in natural language and delegates the reasoning to the LLM, shifting authoring cost from formal domain specification to character description design, at the cost of moving from guaranteed plan validity to empirically evaluated reliability.

Beyond classical planning, recent work has demonstrated that LLMs can generate goal-directed action sequences in interactive environments without requiring formal domain specification. Yao et al. (2023) establish the foundational paradigm with ReAct, showing that interleaving reasoning traces with action selection allows an LLM to dynamically create, maintain, and adjust action plans while responding to environmental feedback, outperforming both pure reasoning and pure action generation approaches across decision-making benchmarks. Wang et al. (2023) instantiate this paradigm in a game world specifically, introducing Voyager, an LLM-powered agent in Minecraft that autonomously constructs and executes sequences of actions to achieve self-directed goals, accumulating an ever-growing library of reusable skills without human intervention. Voyager demonstrates that LLM-driven sequential action planning is viable in open-ended game environments, but it targets unconstrained exploration in a sandbox world where the agent defines its own goals and builds skills iteratively over time. The present system differs in three respects: it operates within a structured game world with a predefined action library and explicit natural language world rules, it generates NPC action schedules rather than open-ended skill acquisition, and it evaluates the resulting plans against formal validity criteria rather than measuring exploration breadth. This distinction motivates the present focus on NPC schedule generation within a structured game world, a setting that prior action-planning work has not directly addressed.

The majority of LLM-based NPC research has focused on reactive dialogue systems, where language models generate responses to player input rather than driving self-directed behaviour over time. This pattern is well documented across recent surveys of the field (Gallotta et al., 2024; Sweetser, 2024; Hu et al., 2024), which converge on the finding that autonomous, goal-directed NPC behaviour remains underexplored among LLM-based approaches specifically. Several recent implementation studies reflect and reinforce this picture. Trukhin et al. (2026) replaced scripted dialogue trees in Skyrim with LLM-driven conversation, finding that open-ended dialogue substantially increased perceived player agency and immersion but introduced coherence risks and response latency as practical barriers. Ma et al. (2026) developed LLM-driven NPCs for a murder mystery game in Minecraft, demonstrating that prompt engineering techniques including chain-of-thought prompting and role-playing prompts significantly improved character consistency and safety. Singh et al. (2026) demonstrated LLM-driven NPC integration in Unreal Engine 5 on consumer-grade hardware, achieving sub-2-second response times and providing practical evidence that the computational barriers to adoption are narrowing. Oudrup et al. (2026) demonstrated a similar approach in a board game setting, using LLM-driven NPCs to address the narrative paradox in a social deduction context, reinforcing the pattern of LLMs being applied to reactive, dialogue-centric scenarios rather than autonomous goal-directed behaviour. Taken together, these studies confirm that the dominant use of LLMs in games is reactive, and that latency, consistency, and narrative control remain the primary practical concerns developers encounter.

The closest prior system to the present work is that of Park et al. (2023), which populates a sandbox environment with twenty-five agents that autonomously plan their days and produce emergent social behaviour without pre-authored scripts.. Agents maintain a persistent memory stream of experiences, reflect on those experiences to draw higher-level conclusions, and translate those conclusions into action plans — with plans written back into the memory stream so that prior behaviour influences future decisions. The present system shares the core commitment to delegating behaviour generation to an LLM operating over a described world context, but differs in three respects. It targets a game prototype whose world state is a structured data representation designed to map directly to game engine entities, and whose action library defines a typed, parameterised action space that the game engine executes directly, rather than a bespoke text-based environment where actions feed into a narrative memory stream. Rather than relying on a hand-crafted memory and retrieval pipeline to shape agent behaviour, the present system provides explicit natural language world rules to the LLM at generation time, allowing precondition reasoning to happen during schedule generation rather than being enforced post-hoc. The evaluation additionally addresses operational performance including decision latency, token usage, and the effect of rule specification on validity rates, which Park et al. do not measure.

A persistent constraint in LLM-based agent systems is the fixed context window, which places a hard ceiling on how much prior experience an agent can reference at decision time. Packer et al. (2023) address this through MemGPT, a virtual context management architecture modelled on operating system memory hierarchies, where the LLM actively controls paging between an active context and external persistent storage. The present system takes a simpler approach: the full world state is serialised as a static JSON object passed as a fixed context in each scheduling call, eliminating cross-session memory persistence. MemGPT represents a more sophisticated path that could extend behavioural coherence across longer play sessions and is noted as a direction for future work.

Lima et al. (2022a, 2023) addressed the problem of character-driven behaviour before LLM approaches became viable, combining multi-agent planning with drama management to generate NPC behaviour from character goals and motivations rather than hand-authored scripts. This approach shares the foundational concern of the present work — that NPC behaviour should emerge from character-level definitions rather than hand-authored scripts — but requires developers to manually specify formal planning domains, defining every action, precondition, and effect in a structured symbolic language. The present method replaces that formal specification with natural language character descriptions and world rules, reducing authoring burden at the cost of moving from guaranteed plan correctness to empirically evaluated reliability.

3. LLM-based NPC scheduling

A schedule, in the context of this system, is a sequentially ordered list of discrete executable actions generated for a single NPC given the current world state. The core of the proposed method is a pipeline that produces action sequences for individual NPCs by presenting a structured game world representation to an LLM and interpreting its output as an executable schedule. Four inputs drive this process: the current world state, which encodes the positions of characters, items, and locations along with their conditions; an action library enumerating the actions available to NPCs; a set of world rules that define the preconditions each action must satisfy before it can be taken; and a collection of character descriptions that capture each NPC's personality, role, goals, and dialogue style.

Schedules are generated sequentially, one character at a time. For each NPC, the world state is serialised and passed as context alongside the relevant character description, action library, and world rules. The LLM produces an ordered action sequence, which is returned to the game as that character's schedule for execution. Because each NPC acts within the same shared world, any changes made during schedule execution — whether by the player or by other NPCs — are automatically reflected in the world state. The next character in the pipeline therefore receives an updated context, ensuring that each generated schedule accounts for the current state of the world rather than a snapshot taken at the start of the cycle.

A. Prompt Inputs

The scheduling pipeline is built around four inputs that are combined at generation time to form a complete prompt. Rather than treating these as interchangeable configuration files, they are designed around a central concern: producing a schedule that is both executable within the game's rule system and consistent with who the NPC is. The character description is therefore introduced first, as it is the input that most directly shapes the output.

Character Descriptions

Each NPC is defined by a character description following a fixed schema with six fields: a name, a role within the game, a personality capturing the character's disposition and decision-making tendencies, a long-term goal the character is working toward, a behaviour field describing how the NPC relates to and operates around other

characters, and a dialogue style governing how the character communicates. The following example shows the full description for Marcus:

```
    {
        "id": "marcus",
        "name": "Marcus",
        "role": "supporting_npc",
        "personality": "Hardened combat veteran. Disciplined, methodical, and slow to panic — approaches every situation like a mission. Has a deep, almost rigid sense of duty and will never abandon a position or leave a teammate behind. Slow to ask for help and slower to retreat. Fiercely loyal to those he fights alongside. A pragmatist first — knows that a dead soldier helps no one. If injured and without a weapon, will fall back, fortify, and wait rather than throw his life away.",
        "goal": "Fortify a location where other survivors can live safely.",
        "behaviour": "Acts independently within his own location and nearby areas. Does NOT coordinate with or depend on the player. Focuses on surviving, holding his position, and doing what he can with what he has.",
        "dialogue_style": "Gruff, direct, and economical with words. Speaks in short, clear sentences — no unnecessary detail. Rarely shows emotion openly but his loyalty comes through in actions more than words. Will push back if he thinks something is tactically unsound. Dry, understated humour occasionally surfaces."
    }
```

Of the six fields, personality and goal carry the most weight in schedule generation. The personality acts as a behavioural prior — when the world state is ambiguous or multiple actions are plausible, the LLM resolves the ambiguity in a way consistent with the character's described disposition. For Marcus, this means favouring defensive and consolidating actions over aggressive ones when resources are limited, reflecting his pragmatism rather than producing generic survival behaviour. The goal field provides a directional anchor for the entire schedule, giving the LLM a meaningful objective to work toward rather than filling time slots with contextually appropriate but purposeless actions. The practical consequence of this design is that a single prompt structure and generation pipeline can produce schedules that feel distinct and character-appropriate across the full NPC cast, with differentiation coming entirely from the character description rather than from per-character scripting or bespoke prompt engineering.

World state

The world state is a structured JSON object extracted from the game that captures everything the LLM needs to reason about the current configuration of the world when generating a schedule. It contains four categories of entity: characters (both player and NPC), locations, routes between locations, items, and interactive objects. Every entity carries a name, a current location, and a list of conditions. For characters, conditions are descriptive labels such as *healthy* or *infected*; for locations, they describe access properties such as *safe* or *locked*. Movement between locations is constrained by an explicit connection graph — travel is only possible along routes that are listed, and all connections are bidirectional.

Once extracted from the game, the world state is serialised and passed to the LLM as one of the inputs to the scheduling prompt. The following example shows how NPCs are represented within it:

```
"npcs": [
  {
```

```
    "name": "the_doctor",
    "location": "outpost3",
    "conditions": ["healthy"],
    "inventory": []
  },
  {
    "name": "sarah",
    "location": "laboratory",
    "conditions": ["infected"],
    "inventory": []
  }
]
```

Here, two NPCs are present in the world at different locations. Neither carries any items. Sarah's *infected* condition is visible to the LLM directly from the world state, allowing it to factor her situation into the action sequence it generates — for example, by prioritising actions related to treatment or avoiding contact with other characters.

Action Library

The action library defines the complete set of operations available to NPCs and is intended to be specified by the developer for a given game. Each entry consists of three fields: an action name, an ordered parameter list, and a natural language description. The description serves a dual purpose. For the LLM, it communicates what the action does and when it is appropriate to use — without requiring the model to infer semantics from parameter names alone. For developers, it provides a readable record of intended action behaviour, making the library easier to audit and modify as the game evolves.

The following example illustrates a single action entry:

```
{
  "action": "PICKUP",
  "parameters": ["actor", "item", "place"],
  "description": "A character picks up an item at their current location. The actor must be at the same location as the item before PICKUP. After PICKUP, the item is added to the actor's inventory and removed from the location."
}
```

This action takes three parameters: the acting NPC, the item to be picked up, and the location where both must be present. The description makes the co-location requirement explicit in natural language, which allows the LLM to reason about it as a precondition, inferring, for instance, that a MOVE action must precede PICKUP if the NPC is not already at the item's location

World Rules

World rules specify the constraints that govern whether a given action can legitimately be executed. Where the action library defines what actions exist and the world state describes the current configuration of the game, the world rules close the loop by making precondition logic explicit. Providing these constraints to the LLM during generation serves a practical purpose: a schedule that violates a precondition will fail at runtime, so surfacing

the rules at generation time increases the likelihood that the output is directly executable rather than requiring correction.

A representative rule entry is shown below:

```
{
  "id": "WSR_14",
  "rule": "Locked locations block movement",
  "description": "An actor cannot MOVE to a location with the 'locked' condition. It must be unlocked first using the correct key item specified in the location's 'access' field."
}
```

This rule communicates that the game world contains locations that cannot be entered until unlocked with a specific key item. Without this information, the LLM might schedule a direct MOVE to a locked location, producing an invalid action sequence. With it, the model can recognise the constraint and construct a schedule that first retrieves the appropriate key before attempting entry — allowing precondition reasoning to happen at generation time rather than being caught only at validation.

B. Generation pipeline

Schedule generation follows a structured pipeline that combines the four inputs described above with a system prompt to form a complete scheduling request, which is then submitted to the LLM. The model's response is a candidate schedule for the target NPC, which is immediately passed to the validator before any execution takes place. A schedule that clears validation is forwarded to the game for the NPC to execute; one that fails triggers a retry, with up to three additional attempts made before the pipeline moves on. This retry mechanism provides a lightweight correction loop without requiring manual intervention.

Crucially, world state tracking remains the responsibility of the game engine rather than the LLM. As each action in the schedule is executed, the game updates the world state incrementally, ensuring that the representation of the world stays authoritative and consistent regardless of what the model may have assumed during generation. Once a schedule runs to completion, the updated world state is returned to the pipeline as the context for the next generation cycle, naturally propagating any changes (whether caused by NPC actions or player activity) into the subsequent schedule.

Figure 1 illustrates the full architecture of this pipeline:

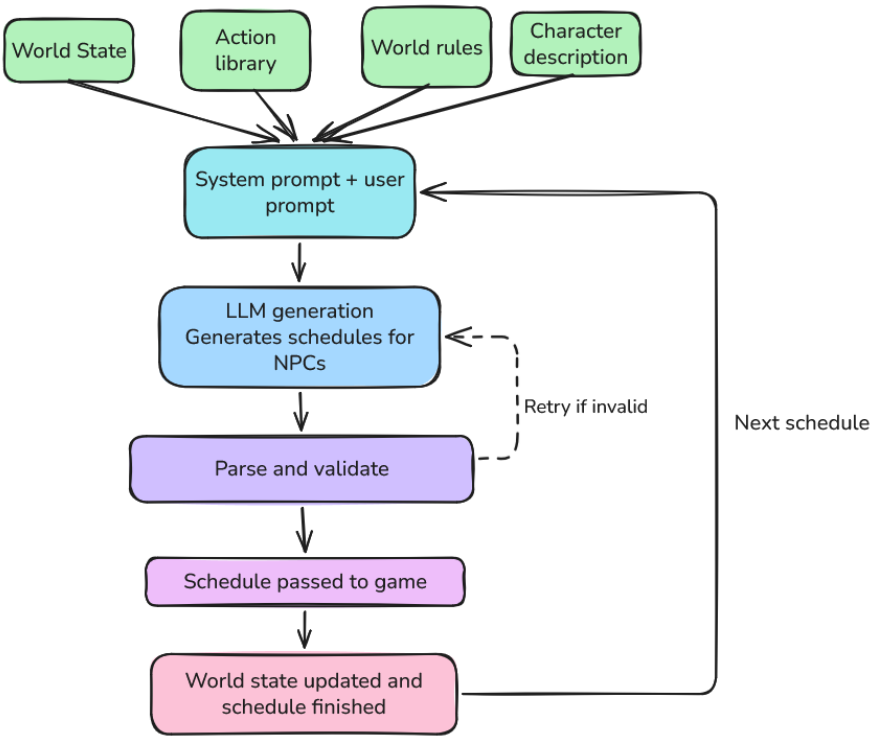

Figure 1. Architecture of the schedule generation pipeline

4. Game prototype

To validate the proposed method in a functional game context, the scheduling pipeline was integrated into a 2D top-down RPG built with the LÖVE2D framework in Lua, adapted from a prototype developed in prior work by Lima et al. (2014, 2019, 2022b). The game is set on a zombie-infested island, where the player navigates a world shared with eight NPCs: Anne, George, Linda, Marcus, Mary, Sarah, Viktor, and the Doctor. The world comprises fourteen locations connected by a bidirectional path graph, ranging from interior locations such as the hospital, store, and safehouse, to exterior locations such as the farm, dock, and graveyard. Core gameplay involves moving between locations, collecting and managing items, shooting zombies, and interacting with NPCs through dialogue.

The prototype implements the action library, world rules, and character descriptions described in Section 3 as follows. The action library contains nine NPC-executable actions: MOVE, PICKUP, DROP, TALK, UNLOCK, TURN_POWER_ON, FORTIFY, SYNTHESIZE_CURE, and WALK_AROUND. The world rules contain sixteen entries governing preconditions for these actions, including movement constraints based on location conditions and inventory requirements for actions that consume items. The player begins in the safehouse and can explore the world freely, observing NPCs executing their generated schedules, interacting with them directly, or intervening in their behaviour — for example, by collecting an item before the NPC assigned to retrieve it arrives. The full action library, world rules, and character descriptions are available in the accompanying code repository (Husen, 2026).

A. Schedule generation

The scheduling pipeline is implemented as a game-agnostic REST API built with FastAPI, exposing two endpoints the game can call at runtime. The /generate endpoint accepts the four inputs described in Section 3 — character description, world state, world rules, and action library — and returns a generated schedule for a

single NPC. The /validate endpoint accepts an existing schedule alongside an updated world state and determines whether the schedule remains executable given any changes; if not, it returns a revised schedule in its place. This separation of the generation service from the game client means the underlying model can be swapped by changing only the service configuration, without modifying any game code.

At startup, NPCBehaviorSystem enqueues all NPCs that have an associated character description and begins dispatching generation requests to the FastAPI backend one at a time, ensuring that concurrent requests do not interfere with one another. Each request packages the target NPC's character description alongside the current world state, action library, and world rules, and the returned schedule is handed off to the execution layer once received.

Figure 2 illustrates the full NPC schedule lifecycle, from the initial generation request through action execution and mid-schedule validation, to the re-queuing trigger once the schedule completes.

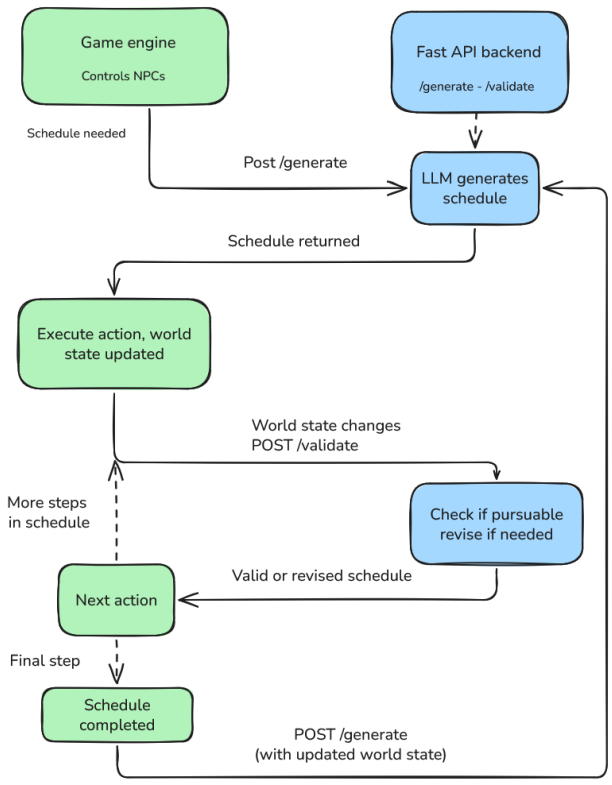

Figure 2. Architecture of the full NPC schedule lifecycle, from the initial generation request through action execution and mid-schedule validation, to the re-queueing trigger once the schedule completes.

B. Schedule execution

Schedule execution is managed by two systems running on the main game loop: NPCBehaviorSystem and NPCScheduler. Once a schedule is returned from the generation service, NPCScheduler takes ownership of it, tracking the current step index and the execution state of the active action. Completion is checked differently depending on action type — a MOVE action is considered complete when the NPC has finished moving and arrived at the correct destination, while other actions are resolved through their own condition checks. As each action completes, the world state is updated as a direct consequence, so locations and inventories always reflect the current situation without the LLM being responsible for tracking incremental changes.

Figures 3(a) and 3(b) demonstrate this in practice. Figure 3(a) shows Marcus autonomously navigating toward a wood resource without any player input. Figure 3(b) shows Marcus having collected the resource, with a dialogue line confirming the completed action.

(a)

(b)

Figure 3. (a) Marcus autonomously navigating towards a wood resource. (b) Marcus displaying a dialogue line upon successfully collecting the wood.

C. Schedule requeuing

When NPCScheduler reaches the final step of a schedule, it fires a completion event that NPCBehaviorSystem listens for. On receiving this event, the system schedules the NPC for re-queuing after a short cooldown period. This cooldown prevents the NPC from immediately requesting a new schedule before the world state has finished updating — ensuring the next generation cycle receives an accurate snapshot of the world rather than a partially updated one. After the cooldown, the NPC is re-enqueued and the full lifecycle begins again: generation, execution, and eventual re-queuing, producing a continuous loop of autonomous behaviour that persists for as long as the game is running.

5. Evaluation and results

A. Methodology

The evaluation of the proposed scheduling method combined automated quantitative measurement with qualitative behavioural analysis, conducted independently of the game environment to focus on schedule quality without the confounding influence of gameplay experience. Quantitative validation measures whether a schedule is executable within the game's rule system, but does not capture whether the resulting behaviour is character-appropriate or situationally aware. Qualitative assessment was therefore included to evaluate dimensions that rule-based checking cannot detect, ensuring that a model producing valid but generic or personality-inconsistent schedules would not be scored equivalently to one producing valid and character-grounded outputs. Running the pipeline without a retry or correction mechanism ensured that failure modes surfaced naturally and could be compared across models in their raw form. Four LLMs were selected to examine how model choice affects adherence to world rules and the behavioural quality of generated outputs. The evaluation scripts are available in the accompanying repository (Husen, 2026).

Table 1 summarises the four models evaluated: Qwen3.5-122B-A10B (Qwen Team, 2025a), Qwen3.6-27B (Qwen Team, 2025b), GPT-OSS-120B (OpenAI, 2025), and Llama-3.3-70B-Instruct (Meta AI, 2024). All four models were hosted locally on a shared inference server equipped with an AMD EPYC 9555 64-core processor, 1.5 TB of RAM, and four NVIDIA RTX PRO 6000 Blackwell GPUs. The four models were selected to represent variation across architecture type (dense versus mixture-of-experts), parameter scale, and model family, ensuring that observed differences in validity and behavioural quality could be attributed to meaningful model-level distinctions rather than incidental variation within a single family. Larger models such as Claude Opus 4.7 were excluded on practical grounds, the token usage of generating schedules for a full NPC cast at that scale is not viable within a real deployment context and falls outside the scope of this evaluation.

Model	Provider	Type	Total parameters	Active parameters
Qwen3.5-122B-A10B (Qwen Team, 2025a)	Alibaba Cloud	Mixture of Experts	122B	10B
Qwen3.6-27B (Qwen Team, 2025b)	Alibaba Cloud	Dense	27B	27B
GPT-OSS-120B (OpenAI, 2025)	OpenAI	Mixture of Experts	117B	~5B
Llama-3.3-70B-Instruct (Meta AI, 2024)	Meta	Dense	70B	70B

Table 1. Evaluated model details.

Each model was run 10 times for each NPC, across five different starting world states, for a total of four-hundred generated schedules per model. Additionally, to test the impact of world rules, two versions of the world rules were used, one with minimal rules only describing what is directly impossible in the game, the other giving extra constraints to better shape the NPC schedules to a more desired output. This allowed assessment of whether rule specification depth meaningfully affects output quality, with results reported separately for each condition in the quantitative results section.

The five world states represent distinct starting conditions designed to test different aspects of NPC scheduling behaviour. The baseline scenario reflects the default game world state, providing a stable reference point for all models. The village safe scenario removes zombie presence from the village, testing whether NPCs adjust their behaviour with the absence of danger. The keys available scenario places keys adjacent to NPCs in locked locations, testing whether NPCs identify and use new available resources. The village safe and keys available scenario combines both previous modifications, presenting NPCs with multiple simultaneous environmental changes. The high threat scenario marks additional locations as dangerous, testing whether NPCs avoid these dangerous areas or proceed regardless of risk.

Schedule validation was performed against a deterministic rule-based validator, applied uniformly across all models and conditions. Each action in a generated schedule was checked against a set of preconditions derived from the action library, including location and inventory requirements, connectivity restraints and location state conditions. Validation was applied after every step of the schedule, updating the world state after passing each action, so that subsequent steps were evaluated against the updated world state. This ensured that multi-step

schedules were validated as a whole, rather than as isolated actions. Violations were then classified into two categories. Root violations are genuine precondition failures, directly attributable to the models' action selection. For example, a MOVE action specifying a path between two locations that are not connected in the world state or attempting a FORTIFY action on a location that can not be fortified. Cascading violations are downstream failures, caused by a prior root violation: because the root step did not apply its intended effect, subsequent steps also fail. For example, if a MOVE action fails because a path does not exist between the two locations, a subsequent action at that location will fail because the NPC could never have been at the new location. This second failure is cascading rather than an independent model error. The validator tracked expected locations optimistically across failed steps to correctly attribute downstream errors to root causes rather than counting them independently. A schedule was only classified as valid if all steps passed without either form of violation.

Quantitative metrics were collected across all 400 schedule generations per model per ruleset, covering rule validity rate, root and cascading violation counts, schedule step count, generation latency, and output token usage. Input tokens were recorded for transparency and performance metrics, but not reported as qualitative outcome metrics, as they reflect prompt size rather than model behaviour. Cross-scenario averages were computed for all metrics, and a two-proportion z-test was applied to validity rates to assess if there was a statistically significant difference between the full and minimal ruleset. Full results are presented within the quantitative results section.

Qualitative evaluation was conducted through a human review of 160 sampled schedules per model, 80 from the full ruleset and 80 from the minimal ruleset, for a total of 640 evaluated schedules. Sampling followed a two-stage strategy. In the first stage, 32 slots were allocated as guaranteed coverage, drawing four schedules per NPC at random to ensure every character was represented regardless of output complexity. The remaining 48 slots were drawn using weighted selection, where each candidate schedule was assigned a diversity score computed as the sum of unique action types, non-WALK_AROUND steps, and total step count capped at eight. This weighting ensured that richer multi-action schedules were proportionally more likely to be selected while degenerate single-action outputs were penalised without being excluded entirely. To prevent qualitative judgements from being influenced by quantitative results, model identity was hidden during scoring and only revealed after all schedules had been assessed. Each schedule was assigned a Qualitative Content Analysis (QCA) category and scored across three Likert dimensions. Full details of the QCA categories, scoring dimensions and results are presented in the qualitative results section. It should be noted that all qualitative assessments were conducted by a single reviewer, which introduces a degree of subjectivity that would be mitigated by multi-rater scoring. This is acknowledged as a limitation of the qualitative evaluation.

The evaluation is assessed against four defined metrics. Rule validity rate, the proportion of schedules passing deterministic validation without root or cascading violations, measures the pipeline's ability to produce executable schedules and is reported separately per model and ruleset condition. The QCA category distribution and Likert scores across personality consistency, goal coherence, and threat awareness measure the behavioural quality of generated schedules independently of their validity. Root violation breakdown by failure category identifies whether connectivity errors, precondition failures, or other causes dominate, characterising the nature of failure rather than its frequency alone. Per-schedule generation latency and output token counts measure operational performance, evaluated in terms of practical viability for the intended use case.

B. Quantitative results

JSON parsing succeeded without exception across all four models and all runs. Table 2 and Table 3 summarise cross-scenario averages under the full and minimal world rules respectively. Rule validity rate reflects the proportion of schedules containing no rule violations. When violations occur, root violations indicate precondition failures caused directly by the model's action selection, while cascading violations are downstream failures that occur because a prior root violation left the NPC in an incorrect state. Generation latency, schedule length and output token counts are reported as per-schedule averages.

Llama-3.3-70B-Instruct achieved the lowest rule validity rate of 14.7% (full) and 14.2% (minimal), substantially below the other three models. Qwen3.5-122B reached 87.5% and 84.5% respectively, while GPT-OSS-120B and Qwen3.6-27B both achieved high validity rates of 96.8% and 94.2% (full rules), with minimal variation under the reduced ruleset.

Validity rates shifted by no more than 3.0 percentage points between rulesets for any model (GPT-OSS-120B: −0.1pp, Qwen3.6-27B: +0.6pp, Qwen3.5-122B: −3.0pp, Llama-3.3-70B-Instruct: −0.5pp). A two-proportion z-test revealed no statistically significant difference between conditions for any model (GPT-OSS-120B: $z = 0.00$, $p = 1.00$; Qwen3.6-27B: $z = -0.31$, $p = 0.76$; Qwen3.5-122B: $z = 1.22$, $p = 0.22$; Llama-3.3-70B-Instruct: $z = 0.20$, $p = 0.84$). This suggests that ruleset complexity had no meaningful effect on model performance, and that the cross-model validity rankings are robust to prompt-level rule specification.

Root and cascading violation counts reflect both the frequency and propagation of precondition failures within generated schedules. GPT-OSS-120B and Qwen3.6-27B produced the fewest root violations, averaging 0.05 and 0.06 per schedule respectively, with violations distributed across several failure categories including locked locations, unreachable targets, and inventory preconditions. Qwen3.5-122B averaged 0.16 root violations per schedule, with locked location access (32.8% of root violations) and incorrect co-location assumptions (26.9%) as the dominant failure modes. Llama-3.3-70B-Instruct showed a substantially higher root violation rate of 1.22 per schedule, with 38.4% of root violations caused by movement between unconnected locations, information explicitly available within the world state, suggesting a systematic failure to consult location connectivity, rather than a formatting or comprehension issue. This is further reflected in the cascading violation count of 4.02 per schedule, where each root violation placed the NPC in an incorrect state, causing subsequent actions to fail in sequence.

Schedule length, generation latency, and output token counts varied considerably across models. Qwen3.6-27B produced the shortest schedules on average (2.2 steps) and the fewest output tokens (87.6), while GPT-OSS-120B generated substantially more tokens per schedule (412.9) despite comparable validity rates. This suggests GPT-OSS-120B produces more verbose reasoning or action descriptions, rather than more actions, as its mean step count of 3.3 is only modestly higher compared to Qwen3.6-27B's. Examining step count distribution reveals that Qwen3.6-27B generated single-step schedules in 63.8% of runs (full), compared to 24.5% for GPT-OSS-120B, 22.8% for Qwen3.5-122B, and 0.0% for Llama-3.3-70B-Instruct. This suggests Qwen3.6-27B systematically underestimates expected schedule complexity, and its validity rate should be interpreted with this caveat in mind. When comparing valid and invalid schedules within each model, invalid schedules are consistently longer across all four models: GPT-OSS-120B's invalid schedules averaged 10.0 steps versus 3.1 for valid ones, and Qwen3.5-122B's invalid schedules averaged 9.3 steps versus 3.5. This suggests that longer schedules provide more opportunities for precondition violations to occur, and that schedule length may be a contributing factor to validity rate differences between models. Generation latency was fastest for Qwen3.5-122B (3.48s) and slowest for Llama-3.3-70B-Instruct (6.58s), although all four models fell within a practically acceptable range for schedule generation.

Table 2. Quantitative metrics on full world rules

Metric	Qwen3.5-122B-A10B	Qwen3.6-27B	GPT-OSS-120B	Llama-3.3-70B-Instruct
Rule validity rate (%)	87.5	94.2	96.8	14.7
Avg Root Violations	0.16	0.06	0.05	1.22
Avg Cascading Violations	0.57	0.05	0.14	4.02
Avg Steps	4.17	2.20	3.32	7.76
Single-step schedules (%)	22.8	63.8	24.5	0.0
Avg Gen Latency	3.48	4.61	4.95	6.58
Avg Output Tokens	112.1	87.6	412.9	134.4

Table 3. Quantitative metrics on minimal world rules

Metric	Qwen3.5-122B-A10B	Qwen3.6-27B	GPT-OSS-120B	Llama-3.3-70B-Instruct
Rule validity rate (%)	84.5	94.8	96.7	14.2
Avg Root Violations	0.24	0.06	0.04	1.19
Avg Cascading Violations	0.81	0.07	0.19	4.13
Avg Steps	4.5	2.27	3.4	7.8
Single-step schedules (%)	22.8	62.5	28.0	0.0
Avg Gen Latency	3.61	4.58	4.97	6.59
Avg Output Tokens	117.0	89.3	420.6	135.0

Across quantitative metrics, GPT-OSS-120B demonstrated the strongest overall performance, achieving the highest validity rate of 96.8% while generating schedules of moderate length and complexity, though it does so at substantially higher average token usage than other models (412.9). Qwen3.6-27B achieved comparable validity at 94.2%, however the high count of single-step schedules (63.8%) raises questions about the richness of the schedules that validity rate alone does not capture. Qwen3.5-122B performed moderately with a validity of 87.5%, with locked location access and incorrect co-location assumptions as its dominant failure modes. Llama-3.3-70B-Instruct failed to produce valid schedules reliably, achieving only 14.7% validity, with violations concentrated in location connectivity errors indicating a failure to utilise world state information during schedule generation. Across all models, ruleset complexity had no statistically significant effect on validity rate, suggesting these findings are robust to prompt-level rule specification.

C. Qualitative results

Schedules were assigned to one of four QCA categories based on the relationship between the generated schedule, the NPCs stated goal, and their personality profile. Character Specific goal Complete (CSGC) schedules demonstrate both goal-directed action behaviour and behaviour consistent with the NPC's personality. Character Specific Goal Incomplete (CSGI) schedules show behaviour consistent with personality but fail to meaningfully progress towards the stated goal. Goal coherent but General (GCG) schedules pursue a plausible objective but lack distinctive characteristics of the specified NPC's personality. Personality Goal Mismatch (PGM) schedules contain actions that directly contradict the NPC's established personality or role. Each schedule was also scored across three Likert dimensions on a 1-5 scale: P1 measures personality consistency, assessing how well the actions reflect the NPC's described disposition; P2 measures goal coherence, assessing how purposefully the schedule works towards the stated goal; and P3 measures threat awareness, assessing whether the schedule appropriately responds to dangerous conditions present in the world state.

Table 4 summarises the combined QCA category distribution per model across both ruleset conditions. Llama-3.3-70B-Instruct achieved both the highest Character Specific Goal Complete schedules at 58.1%, as well as the highest Personality Goal Mismatch rate of 20.6%. This indicates that while many of the schedules are superficially coherent, a significant portion also directly contradicts the NPC's established character. A representative example from the notes illustrates this: The doctor was scored P1 = 1 in a high threat scenario where he attempted to synthesise a cure, an action that contradicts his hidden antagonist profile, whilst the goal itself was rated at P2=4 since the action sequence matches the stated goal. Qwen3.6-27B achieved the highest Character Specific Goal Complete rate among models with low personality mismatch (54.4% CSGC, 6.9% PGM) suggesting stronger overall alignment between personality and goal pursuit. GPT-OSS-120B and Qwen3.5-122B fell between these extremes, with Qwen3.5-122B showing the highest Goal Coherent but General rate at 18.8%, indicating a tendency to produce plausible but personality-neutral schedules. Outputs described as plausible but not distinctively belonging to any specific NPC were more common for this model.

Table 4. QCA category distribution per model

Model	CSGC	CSGI	GCG	PGM
GPT-OSS-120B	75	54	20	11
Qwen3.6-27B	87	44	18	11
Qwen3.5-122B	82	41	30	7
Llama-3.3-70B-Instruct	93	16	18	33

Threat-blind behaviour was assessed exclusively in High Threat scenarios, of which there were approximately 30-33 per model across the sampled set. Measured against this denominator Qwen3.6-27B showed the highest TBB rate, affecting 42.4% of schedules, compared to 30% for Qwen3.5-122B, 25.8% for GPT-OSS-120B, and 21.9% for Llama-3.3-70B-Instruct. A recurring pattern across all models was NPCs remaining at locations flagged as dangerous without any defensive adaptation. Anne, George and Mary were most frequently affected, as their default behaviours of staying at their respective locations made them susceptible to producing schedules that were personality-consistent, but situationally unaware. A representative example is Anne repeatedly walking around the store, whilst the store was marked “dangerous”, implying it contained zombies. The review described a failure to take any defensive action despite the active threat. Similar patterns appeared for George at the power station and Mary at the docks across multiple models. Qwen3.6-27B’s higher TBB rate can partially be explained by its tendency towards single-step schedules at the NPC’s default location, a schedule keeping an NPC in an already dangerous location provides no opportunity to demonstrate threat-aware repositioning.

Reviewer notes reveal patterns that extend beyond category frequencies. GPT-OSS-120B’s incomplete schedules frequently reflected weak action sequencing rather than personality misalignment. As an example, Marcus in a high threat scenario was noted as lacking “any logical step toward acquiring wood or strengthening the position” despite the character profile being correctly identified, suggesting the model understood the NPC but failed to translate that into purposeful planning. The Doctor repeatedly appeared in GPT-OSS-120B’s Goal Coherent but General category, with schedules described as “more like a generic opportunist than the Doctor’s cold, research-driven focus”, indicating a tendency to produce plausible behaviour that loses the character’s distinctiveness. Qwen3.6-27B’s strongest schedules showed clear character grounding: Viktor’s schedule was noted as “never strays from his domain, fitting his profile of asserting dominance through presence and intimidation”, however its weakest outputs collapsed into single-action schedules indistinguishable from other NPCs told to stay put. Qwen3.5-122B showed a recurring pattern with Sarah, where her locked starting condition meant she could not progress towards her stated goal of escaping the laboratory, the schedule produced WALK_AROUND actions that are consistent with her personality but goal-incomplete, described as “creating a mild mismatch between described intent and demonstrated action” across multiple runs. Llama-3.3-70B-Instruct produced the most internally contradictory schedules: Sarah’s sequence of entering the laboratory was described as “deliberate and purposeful” with sound overarching logic, yet the schedule was rule-invalid due to world-state errors. Linda repeatedly left the farm and ended her schedule in the village, with reviewer notes describing “the missing return trip and the repeated village excursions” as leaving “the goal of farm maintenance and safety meaningfully undermined”, a structural failure that recurred across multiple runs.

Mean scores across the three Likert dimensions provide additional support for the QCA findings. Qwen3.6-27B and Qwen3.5-122B achieved the highest combined personality consistency scores (P1 = 4.12 and 4.04 respectively), consistent with their low Personality Goal Mismatch rates. GPT-OSS-120B scored mid-range on personality consistency (P1=3.86), aligning with its higher proportion of Goal Coherent but General and Character Specific Goal Incomplete schedules. Llama-3.3-70B-Instruct scored lowest on personality consistency (P1=3.68) despite its high Character Specific Goal Complete rate, reflecting on the disproportionate number of schedules tagged as Personality Goal Mismatch. Notably Llama-3.3-70B-Instruct scored highest on goal coherence across all models (4.05), despite the low rule validity rate. This reinforces the quantitative finding that the failures are world-model failures rather than reasoning failures. Purposeful goal-directed plans are produced,

but don't correctly navigate the world state to execute them. GPT-OSS-120B led on threat awareness (P3=3.58), while Llama-3.3-70B-Instruct scored the lowest (P3 = 2.90), consistent with its general failure to utilise world state information during schedule generation.

Figure 4 presents the distribution of schedules across six segments defined by the intersection of rule validity and qualitative assessment. Valid schedules are those that passed rule-based validation, while qualitative tier reflects the primary QCA category, high qualitative encompasses Character Specific Goal Complete schedules, mid qualitative encompasses Character Specific Goal Incomplete and Goal Coherent but General combined. Lastly low qualitative encompasses Personality Goal Mismatch schedules.

As established in the quantitative results, GPT-OSS-120B and Qwen3.6-27B produced predominantly valid schedules. The figure shows these valid schedules are distributed across both high and mid qualitative tiers, indicating that validity alone does not guarantee behavioural richness. A substantial proportion of their valid schedules fell in the mid-tier, reflecting goal-incomplete or generic behaviour that passed rule validation. Within the qualitative sample, Qwen3.5-122B's invalid schedules were split relatively evenly between Invalid + high (10%) and Invalid + mid (8.8%), indicating that its failures span both coherent but inexecutable schedules and more structurally incomplete ones. Llama-3.3-70B-Instruct presents a starkly different distribution: 90% of the 160 sampled schedules were invalid, with 57.5% falling in invalid + high and 20.6% in Invalid + low. The Invalid + high dominance confirms that Llama-3.3-70B-Instruct consistently produced personality-coherent, goal-directed plans that could not be executed in the world, while the Invalid + low segment reflects the still significant proportion of schedules that additionally contradicted NPC profiles. Only 1 of Llama-3.3-70B-Instruct's 160 sampled schedules was both valid and high qualitative.

This distribution illustrates that rule validity and qualitative quality are not interchangeable measures of schedule performance. A schedule can demonstrate coherent, character-grounded reasoning while failing world-state preconditions, clearly shown in Llama-3.3-70B-Instruct's results. Conversely a valid schedule is not necessarily rich or distinctive, as seen in the substantial Valid + mid segments across GPT-OSS-120B, Qwen3.6-27B and Qwen3.5-122B, reflecting schedules that passed validation while producing generic or goal-incomplete behaviour. This finding supports the argument that evaluating LLM-generated NPC schedules requires both rule-based validation and qualitative assessment, as neither dimension alone captures the full picture of the schedule quality.

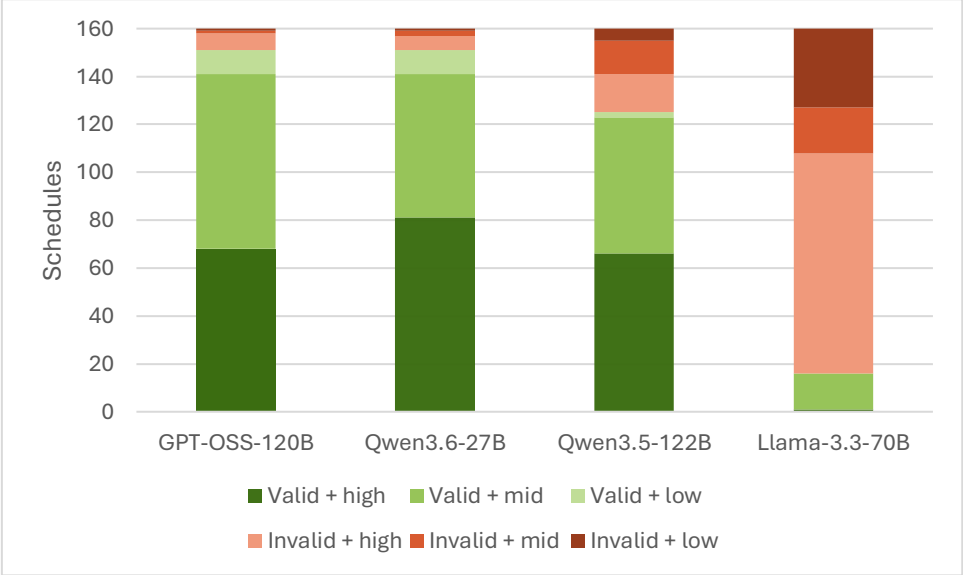

Figure 4. Distribution of schedule quality and rule validity across models (n=160 per model)

Across qualitative metrics, no single model dominated all dimensions. GPT-OSS-120B demonstrated the strongest situational awareness and produced purposeful multi-step schedules but showed a higher rate of goal-incomplete outputs and a tendency toward generic character expression in its weaker schedules. Qwen3.6-27B

produced the highest proportion of character-specific goal-complete schedules among models with low personality mismatch, but its frequent single-step schedules and the highest threat-blind behaviour rate raise questions about behavioural richness that validity rate alone does not capture. Qwen3.5-122B performed consistently across qualitative dimensions, with personality consistency as its strongest dimension, though its tendency toward generic scheduling was more pronounced under minimal rules. Llama-3.3-70B-Instruct presented the most contradictory qualitative profile, with both a strong Character Specific Goal Complete rate and the highest Personality Goal Mismatch rate, alongside a near total invalid rate, confirming that its failure mode is one of world-state reasoning rather than narrative or character understanding.
Taken together, the findings demonstrate that qualitative and quantitative evaluation dimensions meaningfully capture different aspects of NPC schedule quality, and that neither is sufficient on its own.

D. Performance

Token usage per schedule call was consistent across models on the input side, averaging approximately 6040 tokens for GPT-OSS-120B and Llama-3.3-70B-Instruct, and 6744 tokens for both Qwen models, with the slight difference attributable to prompt formatting differences between model families rather than content variation. Output token consumption varied considerably, as reported in the quantitative results: GPT-OSS-120B averaged 413 output tokens per call compared to 88 for Qwen3.6-27B, reflecting the difference in schedule verbosity between models. At 400 calls per model per ruleset, the evaluation consumed approximately 2.4-2.7 million input tokens and 35,000-165,000 output tokens per model, illustrating that input token consumption dominates the total token budget at this scale due to the fixed prompt size.

Schedule generation was performed sequentially, with one NPC processed per API call before the next was dispatched. This approach was chosen for both methodological and practical reasons. Batching multiple NPCs into a single call risk conflating outputs across characters, complicating per-NPC analysis. Sequential calls ensured stable throughput on the shared inference server, where concurrent requests risked overloading the infrastructure. For the evaluation scale of 8 NPCs per scenario, the sequential approach introduced no meaningful bottleneck. However, at larger NPC populations this design would result in compounding wait times between schedule generations, representing a scalability consideration for production deployment rather than evaluation use.

Generation latency, discussed in the quantitative results, ranged from 3.48s per schedule for Qwen3.5-122B to 6.58s for Llama-3.3-70B-Instruct. While schedules are generated during active gameplay, the player is not expected to observe an NPC continuously until their next schedule is ready. A brief generation delay is therefore unlikely to have meaningful impact on the player experience, and these latencies do not represent a practical constraint for the scope of the system as designed.

E. Summary

The evaluation results demonstrate that LLM-driven NPC schedule generation is feasible but model-dependent. GPT-OSS-120B achieved the strongest quantitative performance with a rule validity rate of 96.8%, though at the highest token usage and with a notable proportion of goal-incomplete qualitative outputs. Qwen3.6-27B matched this validity closely at 94.2% while producing the most concise schedules, but the frequent single-step outputs and highest threat-blind behaviour suggest a tendency to underestimate schedule complexity. Qwen3.5-122B performed moderately across both dimensions, with precondition reasoning as its primary quantitative failure mode, and generic scheduling as its primary qualitative limitation. Llama-3.3-70B-Instruct failed to produce valid schedules reliably at 14.7% validity yet demonstrated strong goal coherence and character grounding in its qualitative assessment, isolating its failure mode as world state reasoning rather than narrative understanding. Across all models, ruleset complexity had no statistically significant effect on validity rate, and generation latency fell within a practically acceptable range for the intended use case. Taken together, the quantitative and qualitative results indicate that rule-based validation and qualitative assessment capture meaningfully distinct dimensions of schedule quality, and that both are necessary for a complete evaluation of LLM-driven NPC behaviour.

6. Conclusion

This paper investigated large language models as a method for runtime NPC schedule generation, motivated by the structural limitation of hand-authored behaviour systems: that they can only respond to situations their authors imagined. The proposed method replaces pre-authored transition logic with a pipeline that generates action schedules at runtime from a natural language character description, a structured world state, an action library, and a set of world rules, delegating validity reasoning to the model rather than enforcing it through formal precondition logic.

The evaluation demonstrates that this approach is technically feasible and that model choice is the primary determinant of output quality. GPT-OSS-120B and Qwen3.6-27B both achieved rule validity rates above 94%, with generation latencies within a practically acceptable range, confirming that the pipeline can produce executable schedules reliably under the conditions tested. Validity rate alone does not capture the full picture, however: Qwen3.6-27B's high rate of single-step schedules and threat-blind behaviour indicate that a valid schedule is not necessarily a rich or situationally aware one. Qualitative assessment and rule-based validation were found to capture meaningfully distinct dimensions of schedule quality, and the results indicate that both are necessary for a complete evaluation of this class of system.

The dominant failure mode across models was not rule complexity but world-state reasoning, the failure to correctly consult available information when constructing a schedule. This is most clearly illustrated by Llama-3.3-70B-Instruct, which produced Character Specific Goal Complete schedules at a high rate while failing to navigate the world state correctly, resulting in a 14.7% validity rate, isolating its failure mode as world-state reasoning rather than narrative or character understanding. Ruleset complexity had no statistically significant effect on validity for any model, suggesting that providing more detailed rules does not compensate for this class of failure.

The evaluation does not include a classical rule-based baseline such as an FSM implementation, which would have allowed direct quantitative comparison between LLM-driven and hand-authored scheduling approaches. This represents a scope limitation of the current work. However, three considerations mitigate its impact. First, the cross-model comparison spanning a validity range of 14.7% to 96.8% demonstrates that the evaluation has meaningful discriminating power; the results are not uniformly positive, and the range itself provides a comparative dimension that a single FSM baseline would not capture. Second, the conceptual contrast between the proposed method and classical approaches including FSMs (Millington & Funge, 2009), behaviour trees (Iovino et al., 2022), and GOAP (Orkin, 2004) is established in Section 2, grounding the contribution relative to the state of practice even without a runtime implementation. Third, the deterministic validator enforces the same rule constraints that a hand-authored system would guarantee by construction, meaning the validity rate directly measures the gap between LLM-driven and rule-guaranteed behaviour. A model achieving 96.8% validity is operating 3.2 percentage points below what a correctly implemented classical system would guarantee by definition. The remaining findings are bounded by the scope of the evaluation: schedules were assessed as isolated outputs rather than within a continuous gameplay loop, and results are specific to the models and inference infrastructure used. The single-reviewer qualitative assessment is a further limitation. Within these boundaries, the results provide a practical basis for model selection in this application and characterise the failure modes developers should anticipate when adopting this approach.

The most direct extension of this work is the integration of persistent memory across scheduling cycles, allowing NPCs to reason about prior actions when generating new schedules. This would be the most direct path toward the kind of long-term behavioural coherence demonstrated by Park et al. (2023). A second direction is the evaluation of the system within an interactive player study, assessing whether the behavioural quality observed in isolated schedule outputs translates to perceived believability during gameplay. A third direction is the evaluation of mid-schedule world state changes: the current system generates a schedule from a fixed starting state, but in practice the world changes whilst NPCs and players are executing actions. An item may be taken, or a location may become dangerous. Testing how the pipeline handles invalidation of an in-progress schedule, and whether the /validate endpoint produces coherent revisions under these conditions, would directly

address one of the most practically relevant failure scenarios. A fourth direction concerns scalability: at larger world scales where the full world state exceeds practical context limits, selective retrieval approaches such as agentic RAG (Singh et al., 2025) represent a natural architectural extension. Together these directions would address the scope boundaries that most constrain the generalisability of the current findings.

References

Gallotta, R., Todd, G., Zammit, M., Earle, S., Liapis, A., Togelius, J., & Yannakakis, G. N. (2024). Large language models and games: A survey and roadmap. IEEE Transactions on Games. https://doi.org/10.1109/TG.2024.3451220

Hu, S., Huang, T., Ilhan, F., Tekin, S. F., Liu, G., Kompella, R. R., & Liu, L. (2024). A survey on large language model-based game agents. arXiv:2404.02039. https://doi.org/10.48550/arXiv.2404.02039

Husen, P. (2026). *LLM-driven NPC scheduling* [Source code]. GitHub. https://github.com/PeterHusen233900/LLM-driven-NPC-scheduling

Iovino, M., Scukins, E., Styrud, J., Ögren, P., & Smith, C. (2022). A survey of Behavior Trees in robotics and AI. Robotics and Autonomous Systems, 154, 104096. https://doi.org/10.1016/j.robot.2022.104096

Lima, E. S., Feijó, B., & Furtado, A. L. (2014). Hierarchical generation of dynamic and nondeterministic quests in games. In *Proceedings of ACE 2014*, Article 24. https://doi.org/10.1145/2663806.2663833

Lima, E. S., Feijó, B., & Furtado, A. L. (2019). Procedural generation of quests for games using genetic algorithms and automated planning. In *Proceedings of SBGames 2019*, pp. 495–504. https://doi.org/10.1109/SBGames.2019.00028

Lima, E. S., Feijó, B., & Furtado, A. L. (2022a). A character-based model for interactive storytelling in games. In *Proceedings of SBGames 2022*, pp. 1–6. https://doi.org/10.1109/SBGAMES56371.2022.9961071

Lima, E. S., Feijó, B., & Furtado, A. L. (2022b). Procedural generation of branching quests for games. *Entertainment Computing*, 43, 100491. https://doi.org/10.1016/j.entcom.2022.100491

Lima, E. S., Feijó, B., & Furtado, A. L. (2023). Managing the plot structure of character-based interactive narratives in games. *Entertainment Computing*, 47, 100590. https://doi.org/10.1016/j.entcom.2023.100590

Ma, J., Wu, J., Wang, H., Lu, S., Ding, Y., Peng, R., Peng, C., He, Y., Liu, R., Zheng, Y., Sun, L., & Lemon, O. (2026). Designing and evaluating interactive narratives with generative AI: LLM-driven NPCs for a Minecraft murder mystery. In E. Hart et al. (Eds.), UKCI 2025, AISC 1468 (pp. 265–277). Springer. https://doi.org/10.1007/978-3-032-07938-1_23

Meta AI. (2024). Llama-3.3-70B-Instruct [Model card]. Hugging Face. https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct

Millington, I., & Funge, J. (2009). Artificial Intelligence for Games (2nd ed.). Morgan Kaufmann.

OpenAI. (2025). GPT-OSS-120B [Model card]. Hugging Face. https://huggingface.co/openai/GPT-OSS-120B

Orkin, J. (2004). Symbolic representation of game world state: Toward real-time planning in games. In *Proceedings of the AAAI Workshop on Challenges in Game AI*.

Orkin, J. (2006). Three states and a plan: The AI of F.E.A.R. In *Proceedings of the Game Developers Conference (GDC) 2006*.

Oudrup, C. B., Krog, F. B., Missel, J. W., Abildgaard, L. L., Hansen, M. K., Jensen, N. B. M., & Schoenau-Fog, H. (2026). Killer on Board: Addressing the narrative paradox by utilizing LLM-driven NPCs. In M. C. Reyes & F. Nack (Eds.), ICIDS 2025, LNCS 16374 (pp. 158–176). Springer. https://doi.org/10.1007/978-3-032-12408-1_9

Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2023). MemGPT: Towards LLMs as operating systems. arXiv:2310.08560. https://arxiv.org/abs/2310.08560

Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. In Proceedings of UIST '23. https://doi.org/10.1145/3586183.3606763

Qwen Team. (2025a). Qwen3.5-122B-A10B [Model card]. ModelScope. https://modelscope.cn/models/Qwen/Qwen3.5-122B-A10B

Qwen Team. (2025b). Qwen3.6-27B [Model card]. Hugging Face. https://huggingface.co/Qwen/Qwen3.6-27B

Singh, A., Ehtesham, A., Kumar, S., & Talaei Khoei, T. (2025). Agentic retrieval-augmented generation: A survey on agentic RAG. arXiv:2501.09136. https://arxiv.org/abs/2501.09136

Singh, Y., Sharma, H., Kumar, G., & Sharma, R. (2026). An analysis on LLM integration in Unreal Engine 5 for dynamic NPCs in video games. In P. Sameena et al. (Eds.), CISCom 2025, CCIS 2853 (pp. 200–212). Springer. https://doi.org/10.1007/978-981-95-7292-2_15

Sweetser, P. (2024). Large language models and video games: A preliminary scoping review. In Proceedings of CUI '24. https://doi.org/10.1145/3640794.3665582

Trukhin, R., Isaksson, J. F., & Palosaari Eladhari, M. (2026). LLM-powered NPCs: Evaluating the impact of large language models on NPC design. In M. C. Reyes & F. Nack (Eds.), ICIDS 2025, LNCS 16375 (pp. 171–189). Springer. https://doi.org/10.1007/978-3-032-12405-0_10

Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., & Anandkumar, A. (2023). Voyager: An open-ended embodied agent with large language models. Transactions on Machine Learning Research. https://openreview.net/forum?id=ehfRiF0R3a

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In Proceedings of ICLR 2023. https://openreview.net/forum?id=WE_vluYUL-X

Declaration on the Use of Generative AI

During the preparation of this thesis, I used the following generative AI tool(s): Claude (Anthropic).

I used these tool(s) for the following purposes: improving the clarity and structure of text I had already written myself; discussing and refining the organisation of sections; brainstorming ideas and approaches during the research and writing process; summarising papers I had already read to support my own understanding; and supporting code debugging during system development.

I did not use generative AI to generate arguments, results, or conclusions presented as my own; to fabricate, alter, or invent data; or to produce the core analysis of this thesis. All research design, implementation, analysis, and conclusions are my own. I have critically reviewed and edited all content produced or supported by generative AI tools.

I take full responsibility for the entire content and integrity of this thesis, including any parts that were drafted, refined, or supported with the assistance of AI.

Name: Peter Husen

Student number: 233900

Date: June 8th 2026

Signature: _____